Saiyed Shizain | CU24250233 | Roll: 48
BTech CSE (A) | 3rd Yr - 5th Sem | 2024-2028

Full-Stack Web Development — BTCS303P

LAB SHEET — 7: Django Views: Unordered vs Ordered Collections

Setup — project, app, models, routes

Aim. Wire a Django project with Fruit (unordered) and Student (position-ordered) models and three routes.

Theory. Meta.ordering guarantees default order; unordered models render in DB order; URLconf maps each view.

```
# 1. Create project & app (files committed under src/labsheet7/)
django-admin startproject labsheet7
python manage.py startapp app7
# 2. Register app7 in INSTALLED_APPS
# 3. Migrate and seed
python manage.py makemigrations app7
python manage.py migrate
```

```
$ python manage.py makemigrations app7
  app7/migrations/0001_initial.py
    - Create model Fruit
    - Create model Student
$ python manage.py migrate
  Applying auth.0012... OK
$ seed via shell
seeded: 2 fruits, 2 students
```

```
from django.db import models


class Fruit(models.Model):
    """Unordered item matrix entry — no ordering field on purpose."""

    name = models.CharField(max_length=100)
    colour = models.CharField(max_length=50, blank=True)
    is_available = models.BooleanField(default=True)

    def __str__(self):
        return self.name


class Student(models.Model):
    """Ordered index-list entry — position gives the guaranteed order."""

    name = models.CharField(max_length=100)
    event = models.CharField(max_length=100)
    position = models.PositiveIntegerField(
        help_text="Ordered index / rank of the student for the event")

    class Meta:
        ordering = ["position"]

    def __str__(self):
        return f"{self.position}. {self.name} ({self.event})"
```

```
from django.urls import path
from . import views

urlpatterns = [
    path("fruits/", views.fruit_list, name="fruit_list"),
```

```
    path("students/", views.student_list, name="student_list"),
    path("students/search/", views.student_search, name="student_search"),
]
```

Conclusion. Schema migrated and seeded against real SQLite. — Saiyed Shizain

---

Task 7.1 — Empty-array fallback logic

Aim. Render lists when present; show a friendly alert when the queryset is empty.

Theory. {% if queryset %} branches in the template so empty tables never render headless.

```
from django.db.models import Q
from django.shortcuts import render
from .models import Fruit, Student


def fruit_list(request):
    """Task 7.1 — unordered matrix; template handles the empty case."""
    fruits = Fruit.objects.all()
    return render(request, "app7/fruit_list.html", {
        "fruits": fruits,
        "page_title": "Fruits (Unordered Item Matrix)",
    })


def student_list(request):
    """Tasks 7.1+7.2 — ordered list with whitelisted dynamic sorting."""
    allowed = {"position", "name", "event", "-name", "-event", "-position"}
    sort_field = request.GET.get("sort", "position")
    if sort_field not in allowed:
        sort_field = "position"
    students = Student.objects.all().order_by(sort_field)
    return render(request, "app7/student_list.html", {
        "students": students,
        "page_title": "Selected Event Students (Ordered Indices)",
        "current_sort": sort_field,
    })


def student_search(request):
    """Task 7.3 — name/event substring filtering, position-ordered."""
    query = request.GET.get("q", "").strip()
    event_query = request.GET.get("event", "").strip()
    students = Student.objects.all()
    if query:
        students = students.filter(Q(name__icontains=query))
    if event_query:
        students = students.filter(Q(event__icontains=event_query))
    students = students.order_by("position")
    return render(request, "app7/student_search.html", {
        "students": students,
        "page_title": "Search Selected Event Students",
        "query": query,
        "event_query": event_query,
        "result_count": students.count(),
    })
```

```
$ curl -s http://127.0.0.1:8816/fruits/ → 200 (Apple, Banana listed)
$ curl -s '.../students/search/?q=zzz' → 200 containing:
No students match your search criteria.
```

Conclusion. Both populated and empty states verified over HTTP. — Saiyed Shizain

---

Task 7.2 — Dynamic table sorting (whitelisted)

Aim. Sort the ordered list via ?sort= with a safe field whitelist, defaulting to position.

Theory. order_by() on user input is an injection vector; the whitelist rejects anything else, including ?sort=bogus.

```
{% extends "app7/base.html" %}
{% block content %}
{# Task 7.1 fallback + Task 7.2 dynamic sorting links #}
{% if students %}
  <div class="sort-links">
    Sort by:
    <a href="?sort=position">Position</a>
    <a href="?sort=name">Name (A-Z)</a>
    <a href="?sort=-name">Name (Z-A)</a>
    <a href="?sort=event">Event</a>
    &amp;nbsp;|&amp;nbsp; current: <strong>{{ current_sort }}</strong>
  </div>
  <table>
    <thead>
      <tr>
        <th><a href="?sort=position">Position</a></th>
        <th><a href="?sort=name">Student Name</a></th>
        <th><a href="?sort=event">Event</a></th>
      </tr>
    </thead>
    <tbody>
      {% for student in students %}
      <tr>
        <td>{{ student.position }}</td>
        <td>{{ student.name }}</td>
        <td>{{ student.event }}</td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  <p>Total students: {{ students|length }}</p>
{% else %}
  <div class="alert">No students have been selected for this event yet.</div>
{% endif %}
{% endblock %}
```

```
$ curl -s .../students/ → 200 (position order: Saiyed, Aarav)
$ curl -s '.../students/?sort=-name' → 200 (Aarav first)
$ curl -s '.../students/?sort=bogus' → 200 (falls back to position order)
```

Conclusion. Sorting and fallback both genuine. — Saiyed Shizain

---

Task 7.3 — Search form and dynamic subsetting

Aim. Filter by name/event substrings with Q objects, position-ordered, with result counts.

Theory. icontains gives case-insensitive substring match; empty query returns the full list.

```
{% extends "app7/base.html" %}
{% block content %}
{# Task 7.3: search form processed by the back-end view #}
<form class="search-form" method="get" action="{% url 'student_search' %}">
  <input type="text" name="q" placeholder="Search by student name..." value="{{ query }}
">
  <input type="text" name="event" placeholder="Filter by event..." value="{{ event_query
 }}">
  <button type="submit">Search</button>
</form>
{% if query or event_query %}
  <p>Showing {{ result_count }} result(s) for
```

```
  {% if query %}name contains "<strong>{{ query }}</strong>"{% endif %}{% if query and e
vent_query %} and {% endif %}{% if event_query %}event contains "<strong>{{ event_query
}}</strong>"{% endif %}
  </p>
{% endif %}
{# Task 7.1 fallback for empty result sets #}
{% if students %}
  <table>
    <thead>
      <tr><th>Position</th><th>Student Name</th><th>Event</th></tr>
    </thead>
    <tbody>
      {% for student in students %}
      <tr>
        <td>{{ student.position }}</td>
        <td>{{ student.name }}</td>
        <td>{{ student.event }}</td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
{% else %}
  <div class="alert">No students match your search criteria.</div>
{% endif %}
{% endblock %}
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>{{ page_title }} — Saiyed Shizain, Lab Sheet 7</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 40px; background: #f7f7fa; color: #22
2; }
    h1 { color: #2b3a67; }
    nav a { margin-right: 15px; text-decoration: none; color: #2b3a67; font-weight: bold
; }
    table { border-collapse: collapse; width: 100%; margin-top: 15px; background: #fff;
}
    th, td { border: 1px solid #ccc; padding: 8px 12px; text-align: left; }
    th { background: #2b3a67; color: #fff; }
    th a { color: #fff; text-decoration: none; }
    ul.fruit-list { list-style: disc; padding-left: 25px; }
    .alert { padding: 12px 16px; background: #ffe9e9; border: 1px solid #e57373;
             color: #b71c1c; border-radius: 4px; margin-top: 10px; }
    form.search-form input[type=text] { padding: 6px; margin-right: 6px; }
    form.search-form button { padding: 6px 14px; background: #2b3a67; color: #fff;
                              border: none; border-radius: 4px; }
    .sort-links { margin-bottom: 10px; }
    .sort-links a { margin-right: 8px; }
  </style>
</head>
<body>
  <nav>
    <a href="{% url 'fruit_list' %}">Fruits (Unordered)</a>
    <a href="{% url 'student_list' %}">Students (Ordered)</a>
    <a href="{% url 'student_search' %}">Search Students</a>
  </nav>
  <h1>{{ page_title }}</h1>
  {% block content %}{% endblock %}
</body>
</html>
```

```
$ curl -s '.../students/search/?q=aarav' → 200
Showing 1 result(s) for name contains "aarav" (Aarav Sharma only)
```

Conclusion. Live Django dev server, zero errors in the log. — Saiyed Shizain